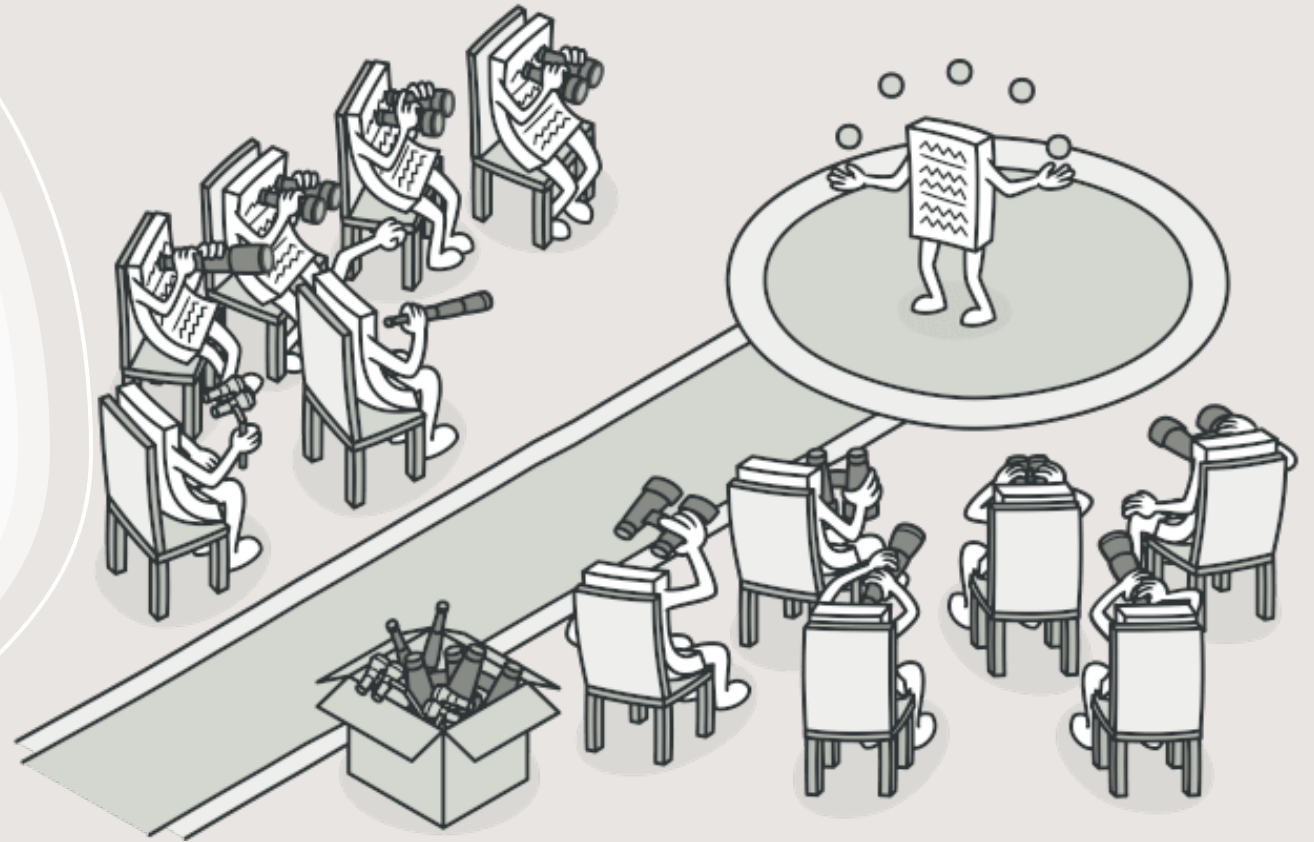
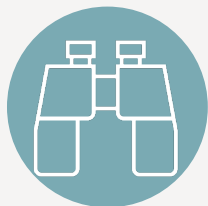


Observer

Pozorovatel





Hlavní idea

Observer (pozorovatel) je behaviorální návrhový vzor (vzor související s chováním), který nám umožňuje definovat mechanismus odběru pro upozornění více objektů na jakékoli události, které se stanou objektu, který pozorují.

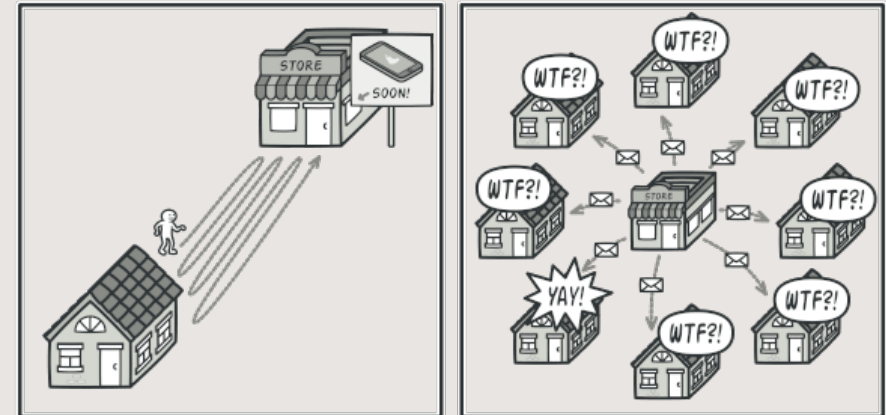
Návrhový vzor Observer se s výhodou využívá v situacích, kdy požadujeme, aby nějaké objekty dostávali informaci o změně stavu jiného objektu a zároveň využívat možnost za běhu ovlivňovat, které objekty to budou.



Problém

Představte si, že máte dva typy objektů: a **Customer** (zákazník) a **Store** (obchod). Zákazník má velký zájem o konkrétní značku produktu (řekněme, že je to nový model iPhone), který by měl být v obchodě velmi brzy dostupný.

Zákazník by mohl navštívit prodejnu každý den a zkontrolovat dostupnost produktu. Ale zatímco je produkt stále na cestě, většina těchto cest by byla zbytečná.





Problém

Na druhou stranu by obchod mohl posílat spoustu e-mailů (které by mohly být považovány za spam) všem zákazníkům pokaždé, když je k dispozici nový produkt. To by některé zákazníky ušetřilo od nekonečných cest do obchodu. Zároveň by to však naštvalo ostatní zákazníky, kteří nemají zájem o nové produkty.

Vypadá to, že máme konflikt. Buď zákazník plýtvá časem kontrolou dostupnosti produktu, nebo obchod plýtvá zdroji upozorňováním nesprávných zákazníků.



Řešení

Objekt, který má nějaký zajímavý stav, se často nazývá **subject** (*předmět*) , ale protože bude o změnách svého stavu upozorňovat i jiné objekty, budeme jej nazývat ***publisher*** (*vydavatel*).

Všechny ostatní objekty, které chtějí sledovat změny stavu vydavatele, se nazývají ***subscribers*** (*odborníci*).

Návrhový vzor Observer navrhuje, abyste do třídy vydavatele přidali mechanismus odběru tak, aby se jednotlivé objekty mohly přihlásit k odběru/odhlásit odběr událostí přicházejících od daného vydavatele. Neboj se! Všechno není tak složité, jak to zní.



Řešení

Ve skutečnosti se tento mechanismus skládá z

1. kolekce pro uložení seznamu odkazů na objekty předplatitele a
2. několika veřejných metod, které umožňují přidávat/odebírat předplatitele do/z tohoto seznamu/kolekce





Řešení

Nyní, kdykoli se u vydavatele stane důležitá událost, projde své odběratele a zavolá konkrétní metodu upozornění na jejich objekty.

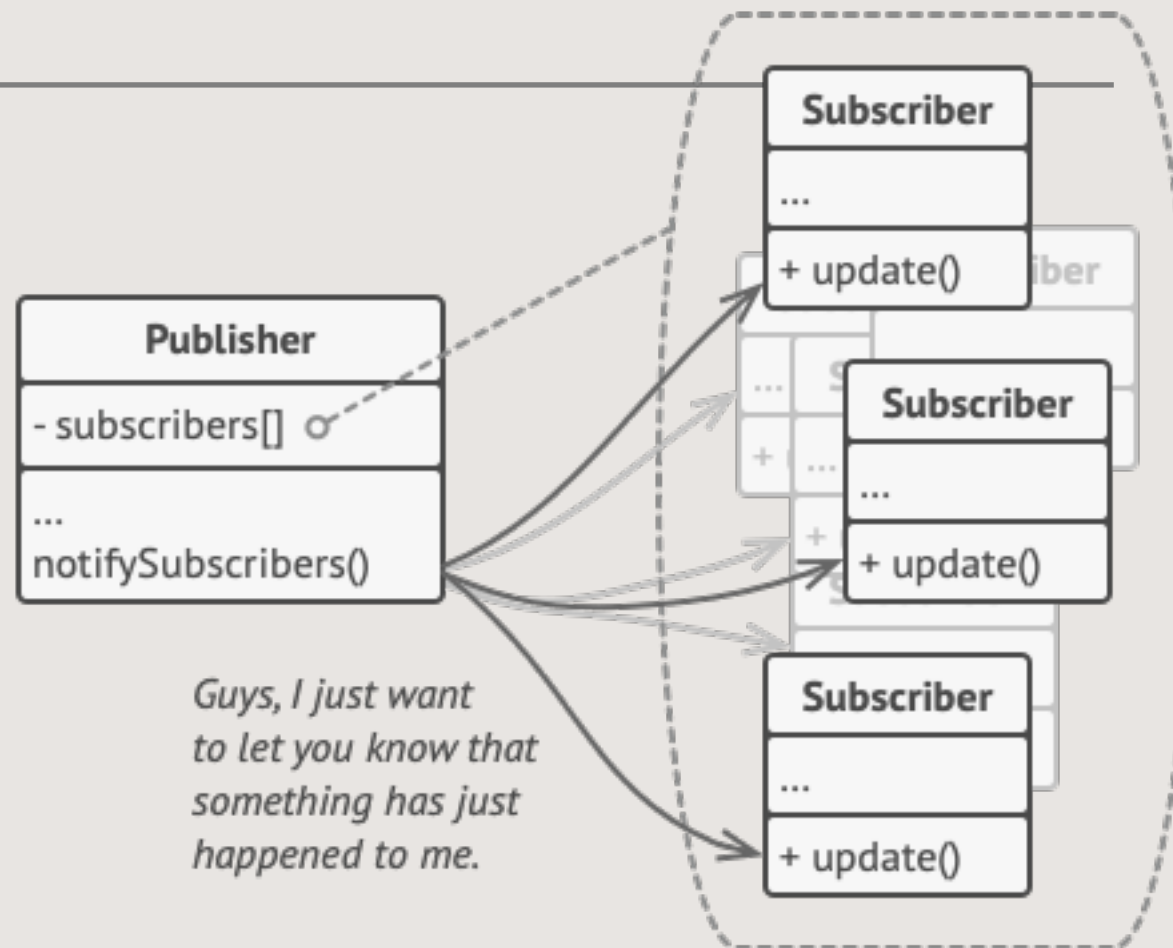
Skutečné aplikace mohou mít desítky různých tříd předplatitelů, kteří se zajímají o sledování událostí stejné třídy vydavatelů.

Proto je klíčové, aby všichni předplatitelé implementovali stejné rozhraní a aby s nimi vydavatel komunikoval pouze přes toto rozhraní. Toto rozhraní by mělo deklarovat způsob oznámení spolu se sadou parametrů, které může vydavatel použít k předání některých kontextových dat spolu s oznámením.



Řešení

Aplikace Publisher upozorní odběratele voláním konkrétní metody oznámení na jejich objektech.





Řešení

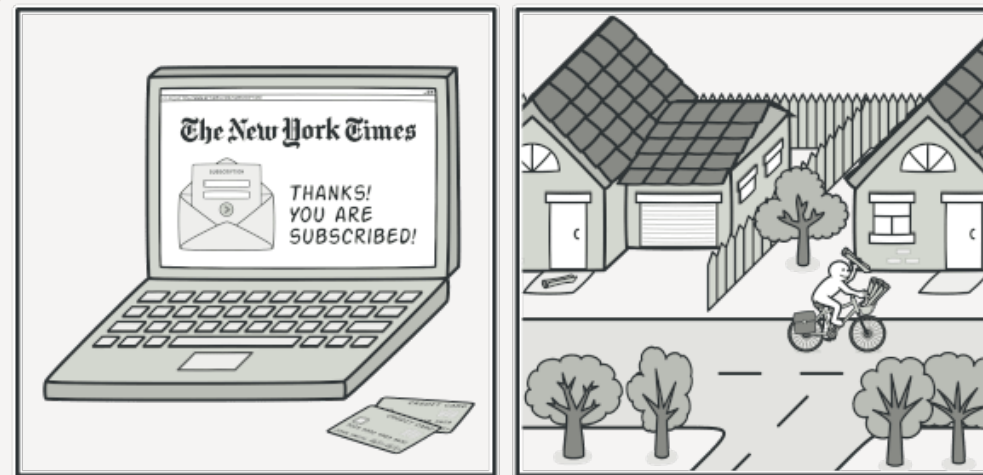
Pokud má vaše aplikace několik různých typů vydavatelů a chcete, aby byli vaši odběratelé kompatibilní se všemi, můžete jít ještě dále a zajistit, aby všichni vydavatelé používali stejné rozhraní. Toto rozhraní by potřebovalo popsat pouze několik metod předplatného. Rozhraní by předplatitelům umožnilo pozorovat stavy vydavatelů bez spojení s jejich konkrétními třídami.



Analogie v reálném světě

Pokud si předplatíte noviny nebo časopis, už nemusíte chodit do obchodu a kontrolovat, zda je k dispozici další číslo. Místo toho vám vydavatel zasílá nová čísla přímo do vaší e-mailové schránky hned po vydání nebo dokonce předem.

Vydavatel vede seznam předplatitelů a ví, o které časopisy mají zájem. Předplatitelé kdykoliv svůj požadavek zrušit, pokud si přejí zastavit zasílání nových čísel časopisu vydavatelem.





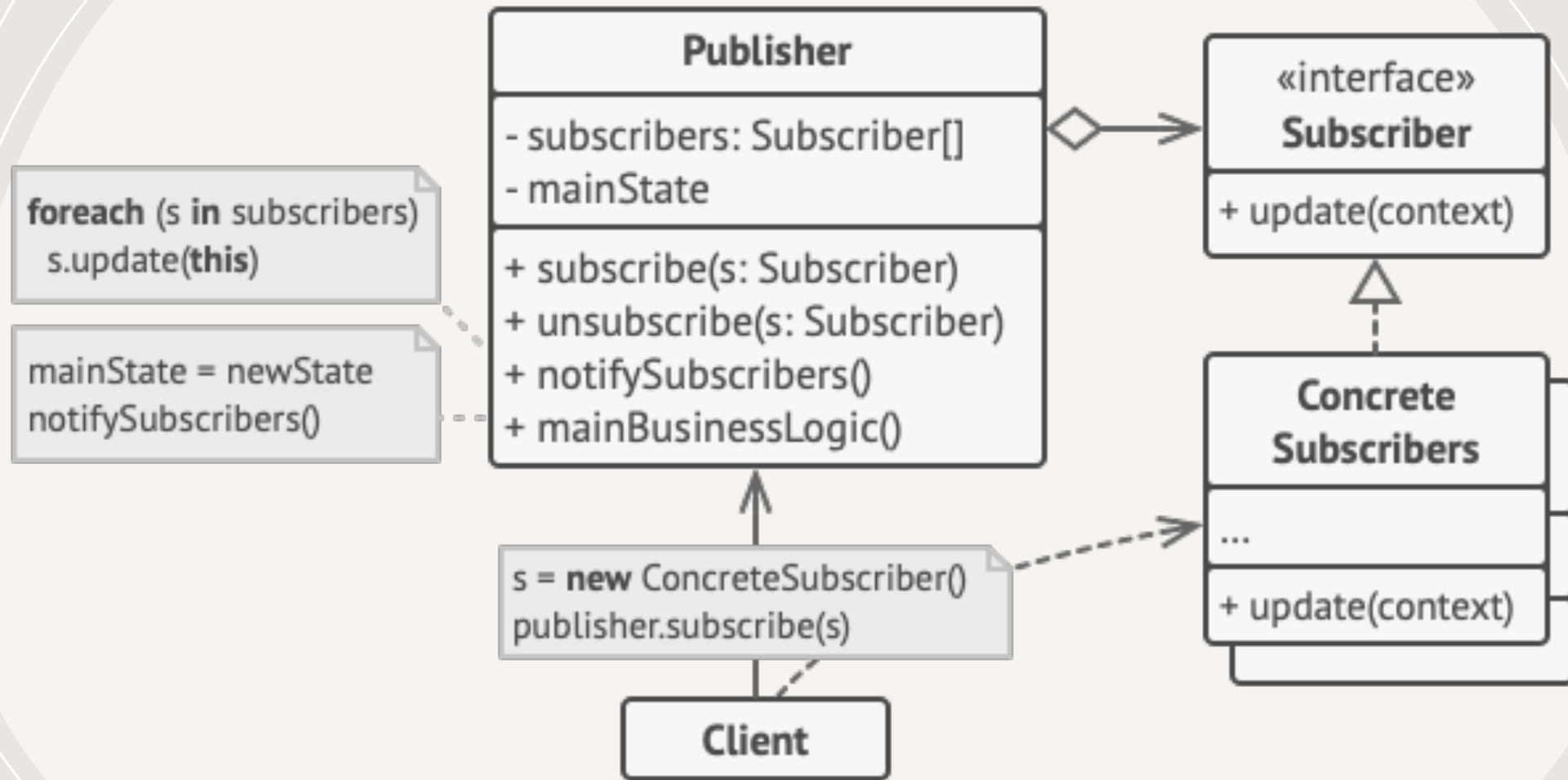
Výhody a nevýhody



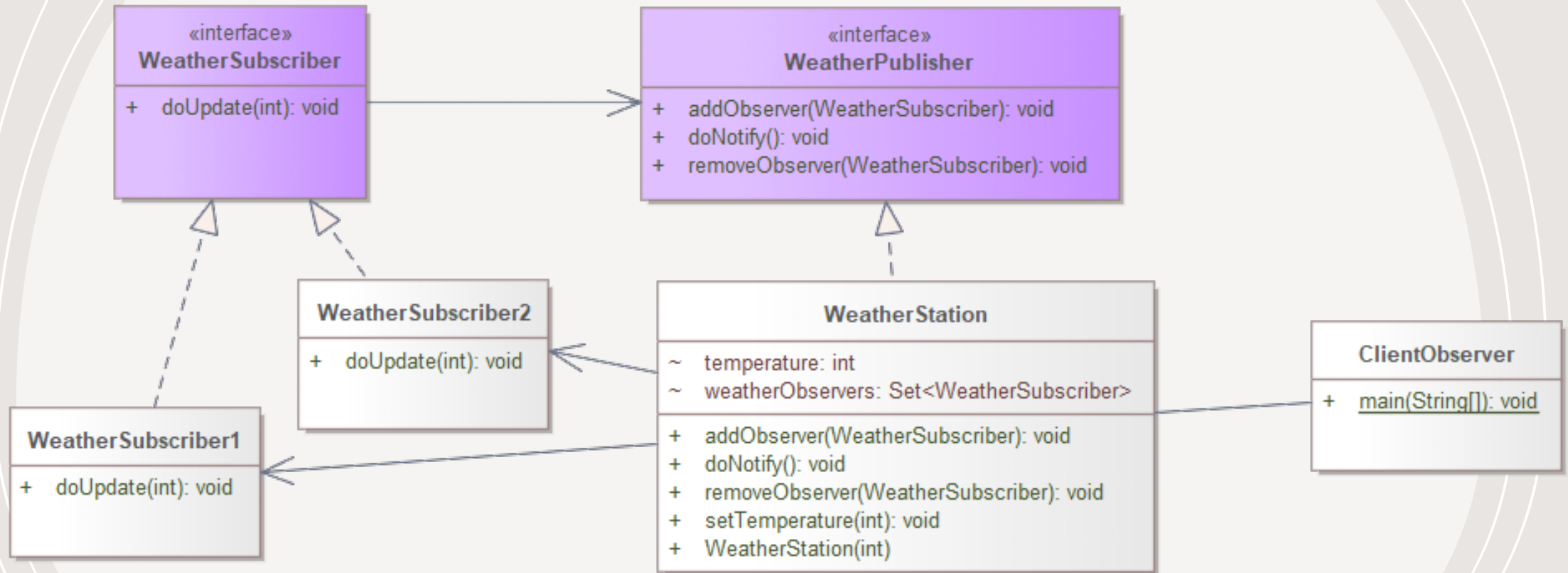
- *Princip otevřený/uzavřený.* Můžete zavést nové třídy odběratelů, aniž byste museli měnit kód vydavatele (a naopak, pokud existuje rozhraní vydavatele).
- Vztahy mezi objekty můžete vytvořit za běhu.



- Předplatitelé jsou informováni v náhodném pořadí.



Struktura



Implementace



```
public interface WeatherSubscriber {  
    public void doUpdate(int temperature);  
}
```

```
public class WeatherSubscriber1 implements WeatherSubscriber {  
    @Override  
    public void doUpdate(int temperature) {  
        System.out.println("Weather customer 1 just found out the temperature is:" + temperature);  
    }  
}
```

```
public class WeatherSubscriber2 implements WeatherSubscriber {  
    @Override  
    public void doUpdate(int temperature) {  
        System.out.println("Weather customer 2 just found out the temperature is:" + temperature);  
    }  
}
```



```
public interface WeatherPublisher {  
    public void addObserver(WeatherSubscriber weatherObserver);  
  
    public void removeObserver(WeatherSubscriber weatherObserver);  
  
    public void doNotify();  
}
```

```
public class WeatherStation implements WeatherPublisher {  
    Set<WeatherSubscriber> weatherObservers;  
    int temperature;  
    public WeatherStation(int temperature) {  
        weatherObservers = new HashSet<WeatherSubscriber>();  
        this.temperature = temperature;  
    }  
    @Override  
    public void addObserver(WeatherSubscriber weatherSubscriber) { weatherObservers.add(weatherSubscriber); }  
    @Override  
    public void removeObserver(WeatherSubscriber weatherSubscriber) { weatherObservers.remove(weatherSubscriber); }  
    @Override  
    public void doNotify() {  
        for(WeatherSubscriber subscriber:weatherObservers){  
            subscriber.doUpdate(temperature);  
        }  
    }  
    public void setTemperature(int newTemperature) {  
        System.out.println("\nWeather station setting temperature to " + newTemperature);  
        temperature = newTemperature;  
        doNotify();  
    }  
}
```



Zdroje

<https://refactoring.guru/design-patterns>

<http://voho.eu/wiki/navrhovy-vzor/>

<https://www.oodesign.com/>